

PROJECT REPORT (PYTHON)

Part A - Origin and Philosophy (Units 1 & 2)

A1. The problem this software was created to solve

In the late 1980s, at a research institute in Amsterdam (Netherlands), called Centrum Wiskunde & Informatica (CWI), a programmer named **Guido van Rossum** found himself stuck between two extremes. He was working on a complex operating system called **Amoeba**, but the tools available to him were frustrating. On one side was **C**, which was powerful but tedious; every small change required recompilation, and even simple tasks demanded **low-level memory management**. On the other side were **Unix shell scripts**, which were quick to write but became messy, slow, and unstructured as projects grew, lacking proper data structures such as **lists and dictionaries**, making programs difficult to maintain as they increased in size.

Guido realised that the programming world needed a **bridge language**, something as easy to read as plain English but as capable as a professional system tool. He drew inspiration from a language he had previously worked on called **ABC**. While ABC was known for its **simple, English-like syntax, indentation-based structure**, and built-in data types, making it perfectly suitable for beginners but it also had significant limitations. It functioned as a **closed system** and could not be easily extend or link or connect with other software such as Amoeba. As a result, despite its simplicity, it was not practical for real-world applications. Guido recognised that the ideal solution would be to combine the **readability of ABC** with the **flexibility and power of C**.

In **December 1989**, during the Christmas holidays when the office was closed, Guido began working on this idea as a personal **“hobby project.”** He named it **Python** not after the snake, but after the British comedy show *Monty Python’s Flying Circus* whose scripts he enjoyed. By **February 1991**, he released the first version, **Python 0.9.0**, on Usenet’s *alt.sources* group. Even at this early stage, Python included advanced features such as **classes with inheritance, exception handling, modules, and core data structures like lists and dictionaries**.

Over the following years, as Python gained popularity, its development continued across multiple institutions. In the mid-1990s, Guido worked on Python at the **Corporation for National Research Initiatives (CNRI)**. In 2000, the core development team moved briefly to **BeOpen.com** and later to Digital Creations (**Zope Corporation**). To ensure that Python remained **open, independent, collaborative and community-driven** with continuous improvement rather than restricting it as proprietary software a **Python Software Foundation (PSF)** was established in **2001**.

The language evolved through several major milestones that shaped its growth. **Python 1.0 (1994)** introduced features such as functional programming tools including **lambda** and **map**. **Python 2.0 (2000)** marked a major expansion with features like list comprehensions, garbage collection, and Unicode support, leading to widespread adoption. However, certain design inconsistencies in Python 2.x led to the development of **Python 3.0 (2008)**, which introduced a cleaner and more consistent structure that forms the foundation of modern Python. It has

become the standard version, continuously evolving with improvements in performance, speed, and usability.

What began as a personal effort to address **practical programming challenges** has since evolved into one of the most widely used programming languages in the world. Python is now widely applied in fields such as **artificial intelligence, scientific research, web development, and automation**. Its success reflects not only its technical strengths but also the **collaborative, open-source community** that has contributed to its continuous development over time.

A2. The license – (The PSF License: A Study in Digital Freedom)

Behind Python's utility lies the **Python Software Foundation (PSF) License**. My audit revealed that this is more than a legal document, it is a contract of trust that powers a global ecosystem.

1. The Core Philosophy: The Four Freedoms :

The idea of open source is often explained using the “Four Freedoms,” originally introduced by Richard Stallman. While working on this audit, I could clearly see how Python supports these freedoms in practice, not just in theory.

- Freedom to Run (Freedom 0):
I was able to run Python on my Ubuntu 24.04 system without any restrictions.
- Freedom to Study (Freedom 1):
Its source code is public. It allows us to understand how the interpreter works.
- Freedom to Redistribute (Freedom 2):
During this project, I uploaded my scripts to GitHub without worrying about legal issues.
- Freedom to Improve (Freedom 3):
I was able to modify scripts and experiment with changes freely according to my need.

2. The PSF License “Grant of Rights” :

The PSF License allows users to use, study, modify, and share Python freely. It encourages deeper involvement, not just usage. Because of this, creating and modifying scripts like `script2.sh` and `script3.sh` felt natural and in line with how the license is designed.

3. The Permissive Nature: Can Companies Sell Python?

Python-based products without sharing their changes. At first, this may seem unfair, but it helps reduce restrictions and makes Python easier to use for everyone- from small startups to large companies. Because of this, Python is widely used across the industry.

4. GPL vs. PSF : A Practical Comparison :

The PSF License is **permissive**, unlike the **GPL (General Public License)**.

- **GPL License:**
Requires that any modified version must also be open source. This protects the community but can be restrictive for businesses.
- **PSF / MIT-style License:**
Allows companies like Tesla or Apple to modify Python and keep their "secret sauce" private.

Personally, I feel that while GPL is good for protecting openness, the PSF approach helps software spread more widely. If I were building my own project, I would probably choose a permissive license for the same reason.

5. “Free as in Beer” vs. “Free as in Freedom” :

Before this project, I mostly thought of “free software” as something that costs nothing. But now I understand the difference more clearly.

- **Free as in Beer (Gratis):** No cost
- **Free as in Freedom (Libre):** Ability to modify and control the software

6. Personal Reflection:

This project changed my perspective about software. Earlier, I mostly saw programs as tools but now, working with Python and Linux (especially on WSL2) made me realize that it is a system I can influence. By writing these scripts, I stopped being a passive consumer and became an active contributor to my own development environment. The PSF License is the reason this transition was possible.

Final Thought:

The PSF License is not just a legal requirement- it’s one of the reasons Python has grown into such a widely used language. By giving users freedom and flexibility, it encourages both learning and innovation.

And after this project, I can say that those freedoms are not just ideas- we actually feel them when we work with open-source software.

A3. The ethics of open source

While the PSF License gives a legal structure, the ethical side of open source is more complex. Open source can be seen as a “digital commons,” like a public library that anyone can use but no one fully owns.

1. The “Free-Rider” Problem:

Large companies like Google and Amazon use Python heavily and make huge profits from it. At the same time, a lot of the work is done by volunteers and the Python Software Foundation.

This arises a question: Is it fair for companies to benefit so much from something that is free? In reality, these companies do contribute by funding projects or adding code, but the effort is not equal. The companies usually gain more than what they give back.

My view: I think companies should support open-source projects because they depend on them. Even small contributions can help avoid problems like burnout and lack of maintenance.

2. Freedom 0 and the “Evil User” Problem

Freedom 0 allows users to use software for any purpose. This is useful, but it also creates a problem. Python can be used for good things like medical research or education but also for harmful things like surveillance or weapons.

So, should developers control how software is used?

Most open-source communities say no. They believe software should stay neutral and it is the user's responsibility to use it properly.

My view: This is a difficult issue. If usage is restricted, it could limit innovation. But if there is no control, misuse is possible. So I think it is better to encourage responsible use rather than strict control.

3. Sustainability of Open Source

Before working on this project, I assumed software simply “works” without much effort. However, while creating scripts like `script4.sh` and analyzing system logs, I realized that maintaining software requires continuous human effort.

Tasks such as: Fixing bugs, updating software, handling security issues, etc., all require time and dedication from developers. If contributions stop, even widely used software can become outdated or insecure. This leads to an important realization:

Users also share responsibility for sustaining open-source software.

Contributions are not limited to coding. They can include: Reporting bugs, writing documentation, helping other users, donating or supporting projects, etc., .

My Reflection:

Through this project, I started seeing myself not just as a user, but as part of the open-source ecosystem. Even small contributions can help maintain and improve these tools.

4. Final Thoughts:

Open source is not just about free access to software- it is about **shared responsibility**.

It allows anyone to use and benefit from software, but ideally, users should also contribute in some way. There are always challenges in balancing:

- Freedom vs control
- Profit vs fairness

- Individual benefit vs community contribution

In my opinion, open source works because of this balance, even though it is not perfect. It has enabled innovation across the world and supports many modern technologies. As future developers, we should not only use open-source tools but also think about how we can support and sustain them.

Part B - Linux Footprint (Unit 2)

Before this project, Python was just something I typed in the terminal. I would write scripts and run them, but I never really thought about where Python actually exists in the system or how it works behind the scenes.

This part made me explore that. I had to check where Python is installed, how it runs, and how Linux manages it. Honestly, it helped me understand Python much better than just coding with it.

1. Installation

Python was already installed on my Ubuntu system:

```
python3 --version # Python 3.12.3
```

Output :- Python 3.12.3

To understand how installation works, I checked the command:

```
sudo apt update  
sudo apt install python3
```

When I ran it, it showed Python was already the latest version.

I also read about installing Python from source (downloading, extracting, compiling, etc.), but it seemed complicated and risky for beginners.

Takeaway: Most users rely on the package manager (apt). It's simple, safe, and efficient.

2. Python File Locations :-

Initially, I thought Python was just one file, but it is actually spread across different directories in the system.

Type	Location	Purpose
Executable	/usr/bin/python3	Main Python command

Standard Library	/usr/lib/python3.12/	Built-in modules (os, json, etc.)
System Packages	/usr/lib/python3/dist-packages/	Installed via apt
User Packages	~/.local/lib/python3.12/site-packages/	Installed via pip
Config Files	/etc/python3/	System configuration

Command used: which python3

This separation ensures system files and user files don't interfere with each other.

Reflection: Python is not a single tool it is organized across the Linux file system in a structured way.

3. Running Python & User Permissions :-

To see how Python runs, I used:

```
ps -ef | grep python
```

This showed that Python runs under the current user.

I also tried:

```
sudo python3 test.py
```

It worked, but then I realized something important running Python as root can be dangerous. If a script has an error, it can affect the whole system.

To avoid this, I used virtual environments:

```
python3 -m venv myenv
source myenv/bin/activate
```

This creates an isolated space where I can install packages safely.

Lesson learned:

- Python runs with user permissions
- Avoid using sudo unless necessary
- Virtual environments are very useful

4. Python as a service :-

At first, I thought Python might run as a service like Apache or MySQL.

I checked:

```
systemctl status python3
```

Output :- Unit python3.service not found

This means Python itself is not a service.

Instead, Python is a **tool used to create services**. If needed, we can run a Python script continuously using systemd.

Takeaway: Python doesn't run on its own it runs only when executed.

5. Updates :-

Python and its packages are updated in two ways:

System Updates :

```
sudo apt update
```

```
sudo apt upgrade
```

Package Updates :

```
pip install --upgrade package_name
```

To check outdated packages :

```
pip list --outdated
```

These updates come from the open-source community.

Reflection: Open source works because people continuously contribute, fix bugs, and improve software.

6. Key Commands :-

Command	Purpose
<code>which python3</code>	Find main Python executable
<code>python3 --version</code>	Check version
<code>ls /usr/lib/python3.12/</code>	Standard library
<code>ls /usr/lib/python3/dist-packages/</code>	System packages

ls ~/.local/lib/python3.12/site-packages/	User packages
`ps -ef	grep python`
sudo python3 test.py	Run script as root
python3 -m venv myenv	Create virtual environment
source myenv/bin/activate	Activate environment
pip list --outdated	Check outdated packages

Reflections :-

- Python is not a single file it is spread across multiple directories
- It runs with user permissions, which can be risky if misused
- Open source is actively maintained, not just an idea
- Virtual environments help in safe development

This section helped me move from just *using* Python to actually *understanding* how it exists inside a Linux system.

It made open source feel practical not just something we study, but something we interact with every time we run a command.

Part C - The FOSS Ecosystem (Units 3 & 4)

Before this part, I mostly used Python only for writing small programs or solving assignments. I never really thought about how it connects with the system or how it is actually used in real scenarios.

In Part B, I explored where Python lives and how it runs. In this part, I focused more on **using Python practically**, creating small scripts, and understanding how it behaves in real use.

1. Writing and Running Python basic scripts :-

```
nano test.py
```

Inside it, I wrote:

```
print("Hello, Python in Linux")
```

Then I ran it using:

```
python3 test.py
```

It worked as expected. This was basic, but it helped me confirm how Python executes scripts from files.

2. Making Python Scripts Executable :-

Instead of running Python every time manually, I learned how to make scripts executable.

First, I added a shebang line at the top:

```
#!/usr/bin/python3
```

Then I gave permission:

```
chmod +x test.py
```

Now I could run it like:

```
./test.py
```

This felt interesting because it behaved like a normal Linux command.

What I learned: Python scripts can act like system-level programs when configured properly.

3. Working with File Handling :-

I created another script to work with files:

```
file = open("data.txt", "w")  
file.write("This is my first file using Python\n")  
file.close()
```

Then I ran it and checked the file:

```
cat data.txt
```

It worked correctly.

Then I modified it to read the file:

```
file = open("data.txt", "r")  
  
print(file.read())  
  
file.close()
```

Reflection: This made me understand how Python interacts with the file system directly.

4. Using Python with system commands :-

I explored how Python can interact with Linux commands.

```
import os  
os.system("ls")
```

This executed the `ls` command from Python.

Then I tried:

```
import os

print(os.getcwd())
```

It showed the current working directory.

What I understood: Python is not separate from Linux it works *with* the system.

5. Automating small tasks :-

I created a small script to automate a task:

```
import os
files = os.listdir()
for file in files:
    print(file)
```

This listed all files in the directory.

Then I modified it slightly to filter `.txt` files:

```
for file in files:
    if file.endswith(".txt"):
        print(file)
```

This felt useful because it saves time compared to doing things manually.

6. Error and Small Mistakes :-

While working, I made a few mistakes:

- Forgot `chmod +x` → script didn't run
- Wrong indentation → Python error
- Misspelled commands → script failed

At first it was frustrating, but fixing them helped me understand how Python behaves.

Lesson learned: Errors are part of learning, especially in scripting.

7. Connecting with Linux Concepts :-

This part connected strongly with what I learned earlier:

- Python uses **Linux permissions**
- Scripts behave like **executables**

- File handling connects to **filesystem structure**
- Commands like `ls` can be controlled using Python

So Python is not just a language it is part of the Linux ecosystem.

8. Learning Outcome :-

From this part, I learned:

- How to create and run Python scripts
- How to make scripts executable
- How Python interacts with files
- How Python can automate tasks
- How errors help in learning

9. Final Reflection :-

This part made Python feel more practical. Earlier, I only used it for coding problems. Now I see how it can be used for real system-level tasks.

It also made me more comfortable using Linux with Python together.

Key takeaway:

Python is not just a programming language it is a powerful tool for interacting with the system and automating real work.

Part D - Open-Source vs Proprietary (Critical Analysis)

Dimension	Python (Open Source)	MATLAB (Proprietary)
Cost	Python is free to use, download, and distribute without any licensing fees. This makes it a good option for students, developers, and even large organizations that want to spend less on the cost but also wouldn't want to give up on the quality.	MATLAB operates on a paid licensing model, and the costs can go up significantly, particularly for individual users or smaller teams. Though student pricing is available, but it is not always accessible, making it a hard unless the institution provides it.
Security	Since Python is open source, its code is open for anyone to review and audit. This type of public transparency means defaults are found easily and can be solved, simply because same code is seen by many people.	MATLAB keeps its source code private, so public can't access it. That is not necessarily a bad, but it does mean users have no real way to verify or check on their own what is happening under the hood.
Support & Reliability	Python has a very active community across various platforms like Stack Overflow, GitHub, etc. There	MATLAB on the other hand has official support along with well-made documentation, which

	are so many resources, but it is all community-driven, which can affect the response times and the quality of work.	makes a big difference in places where quick, reliable answers are necessary.
Freedom to Modify	One thing about Python is that its users can freely modify, adapt, and build upon the source code. Due to this people across different fields like research, industry and education use it.	Working with MATLAB people don't get this freedom and must work within the set boundaries by MathWorks. The tools provided are well-made, but it leaves little room for users to experiment on their own.
Community vs Corporate Control	Python works under the Python Software Foundation along with contributors along the globe. This has led to the language being evolved in ways the users across the world want it rather than what only the company wants.	MATLAB is managed by MathWorks giving no chance to the users to decide anything on their own, the features development to the cost everything decided by the company. Though it gives stability and consistency, but the direction of the software remains solely with the company with no room for new things.
Overall Verdict	Python is flexible, accessible and widely accepted and used by users which makes it a good choice for students, developers, and open-source contributors. It also encourages innovation through freedom and community support.	MATLAB is more structured, so it is good for professional settings that makes it suitable for specific industries, but the cost and lack of flexibility limit its accessibility.

Conclusion :-

After comparing Python and MATLAB across these dimensions, we can see that both have different strengths and weaknesses and neither is a clear winner in every situation. Like for Python the biggest advantages are its openness and accessibility like anyone can download it, use it, modify it, and build on it without worrying about licensing costs or restrictions. This makes it more practical for students and developers who want flexibility without worrying about the cost. But MATLAB cannot be ignored completely. Its official support structure and polished toolboxes make it a very reliable option in industries where things need to work consistently, and help needs to be available when something goes wrong. To be honest both sides have something better of their own.

Still, when thinking about real-world deployment across most contexts like education, research, data science, general software development I think Python is the stronger choice and would recommend that. It does everything most users need, costs nothing, and has a community large enough that someone would help you from somewhere. Also, it contributes in many small ways for the larger community. Fixing a documentation error, sharing a project publicly, answering a question in a forum all these things matter in an open-source ecosystem because the whole model depends on people sharing and helping each other in any way that they can. Python feels less like a tool and more like something people collectively own, and that is a very different relationship to have with software that makes it much more user friendly.

Script 1: System Identity Report - Displays basic system and user information, including **username**, **OS version**, **kernel version**, **uptime**, **home directory**, and **current date/time**. It provides a quick overview of the system environment. It also includes **license information (GPL)** as part of the audit output.

```
lubuntu@DESKTOP-0MDQVFK: $ #!/bin/bash
# Script 1: System Identity Report
# Author: Lubbanti Jain | Course: Open Source Software

# --- Variables ---
STUDENT_NAME="Lubbanti Jain"
SOFTWARE="Python"

KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
DATE_TIME=$(date +%d-%m-%Y %H:%M:%S)
DISTRO=$(grep "PRETTY_NAME" /etc/os-release | cut -d= -f2 | tr -d '"')

# --- Display ---
echo "=====
PYTHON OPEN SOURCE AUDIT
=====
Student Name : $STUDENT_NAME
Software Chosen : $SOFTWARE
=====
Linux Distro : $DISTRO
Kernel Version : $KERNEL
Current User : $USER_NAME
Home Directory : $HOME_DIR
System Uptime : $UPTIME
Date & Time : $DATE_TIME
=====
License Info : GNU General Public License (GPL)"

PYTHON OPEN SOURCE AUDIT
=====
Student Name : Lubbanti Jain
Software Chosen : Python
=====
Linux Distro : Ubuntu 24.04.4 LTS
Kernel Version : 6.6.87-2-microsoft-standard-WSL2
Current User : lubanti
Home Directory : /home/lubbanti
System Uptime : up 9 minutes
Date & Time : 28-03-2026 16:49:03
=====
License Info : GNU General Public License (GPL)
=====
```

Script 2: FOSS Package Inspector- Checks whether a specific open-source software package is installed on the system using system commands. If installed, it shows the version otherwise, it displays an error message. It also provides a short description of the software.

```
lubuntu@DESKTOP-OMDQVFK: ~ % #!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Lubhant Jain | Course: Open Source Software

--- Variable ---
PACKAGE="python3"

echo "=====
echo "      FOSS Package Audit: $PACKAGE"
echo "=====

--- Check if package is installed ---
if dpkg -l | grep -q $PACKAGE; then
echo "[OK] $PACKAGE is installed."
echo "-----"
echo "Version:"
python3 --version
echo "-----"
else
echo "[ERROR] $PACKAGE is NOT installed."
fi

--- Case statement (philosophy note) ---
case $PACKAGE in
python3)
echo "Python: A community-driven language focused on sim
licity and readability."
;;
apache2)
echo "Apache: The web server that built the open interne
t."
;;
mysql*)
echo "MySQL: Open source at the heart of millions of app
esac
;;
*) "FOSS promotes collaboration and transparency." web
=====
FOSS Package Audit: python3
=====
[OK] python3 is installed.
-----
Version:
Python 3.12.3
-----
Python: A community-driven language focused on simplicity and re
adability.
lubuntu@DESKTOP-OMDQVFK: ~ %
```

Script 3: Disk and Permission Auditor- Analyses important system directories and displays their permissions and disk usage (size). It helps in understanding access control and storage usage distribution and also checks for the existence of specific configuration directories like Python config.

